

Large-Scale and Multi-Structured Databases

BookSphere

Danilo Parisi
Matteo Billi Ciani
Matteo Iori

Application Highlights

Book Discovery

- Browse trending books, genre and author.
- Search and rank books by title, author and genre.
- See all reviews of a books.
- See book internationality index.

Registered Users' Experience

- Mark a book either “read” , “reading” or “to-read”.
- Submit books reviews with a vote from 1 to 100, optionally with a written comment.
- Find and Follow other users and like their reviews
- Like an Author, genre or book.
- Get recommendation by the user interaction with the platform.
- Get the Yearly Recap.
- Find Influencers.

Administrative Operations

- Moderate Users' behaviors as delete reviews and ban Registered Users from the platform.
- Add/Update/Delete books and authors in the catalogue.

Actors and main mock-ups (1)



 **Booksphere**

Login

[Sign Up](#)

Your literary life, **organized.**

Explore global reviews, follow real influencers, and discover how far a book travels.

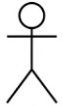
 Search books, authors, or reviews...


Genres

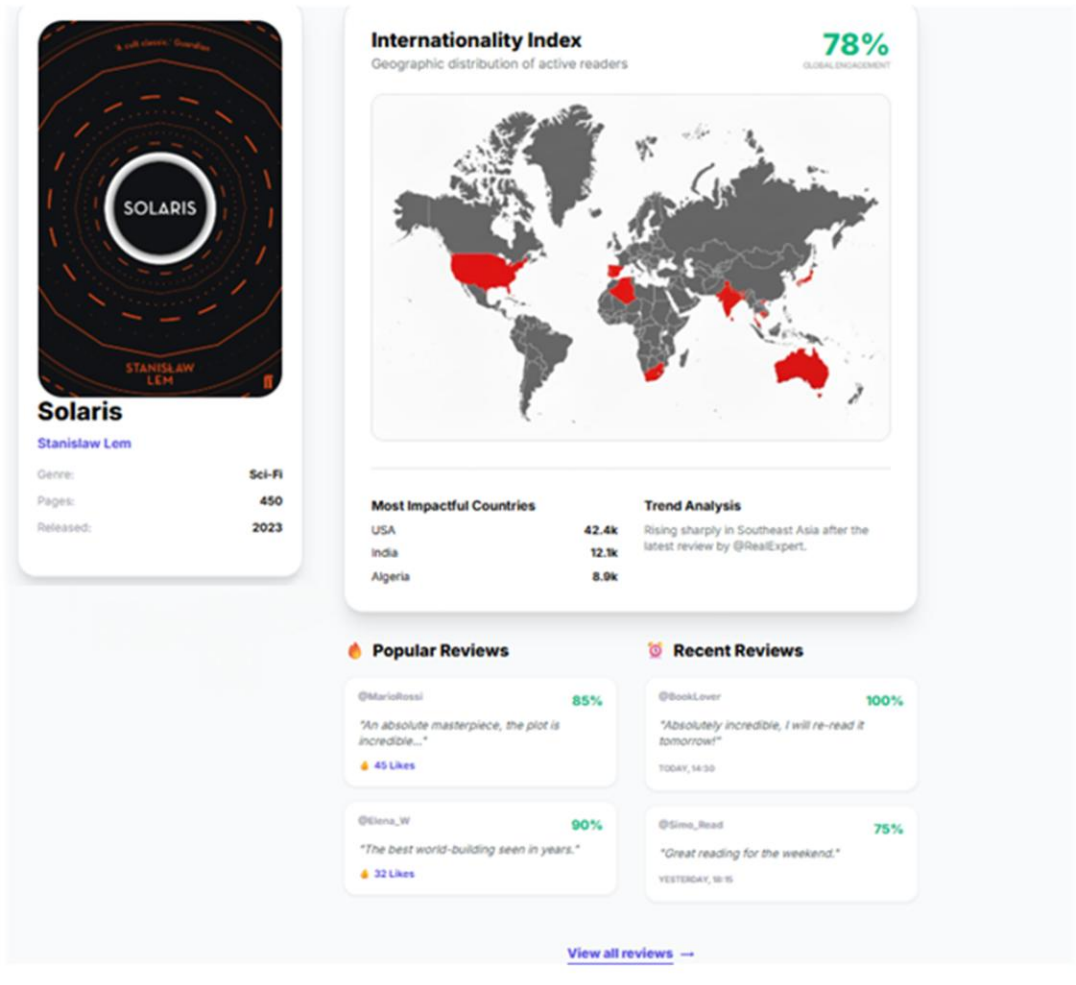

Authors


Trending

Actors and main mock-ups (2)



UNREGISTERED
USER



Actors and main mock-ups (3)



ADMINISTRATOR

AdminSphere
[Dashboard](#)
[Catalog](#)

User Management

View user profiles and manage access.

USER	ROLE	STATUS	ACTIONS
 @alice_r	Registered User	Active	View Profile Ban User
 @spammer_99	Registered User	Banned	View Profile

Review Moderation

Monitor and delete inappropriate reviews.

BOOK	USER	REVIEW	ACTIONS
The Last Dragon	@alice_r	This book was okay but the ending felt rushed...	View Delete
Space Odyssey	@spammer_99	CLICK HERE FOR FREE MONEY!!!	View Delete

Actors and main mock-ups (4)



ADMINISTRATOR

AdminSphere
Dashboard
Catalog

Catalog Management

Insert new content into the Booksphere database.

Add Book

TITLE

AUTHOR NAME

YEAR

SUMMARY

SAVE

Add Author

FULL NAME

REGISTER

Actors and main mock-ups (5)



Dataset Description (1)

Sources: *Amazon Book Reviews* and *Bookcrossing*

Description: Amazon Books Reviews Dataset (~5GB) serves as the core foundation, providing the book catalogue and real-world reviews from Amazon, spanning from 1996 to 2014. Book-Crossing Dataset as a supplementary source, contributing additional user ratings to increase both data variety and volume.

Pre-processing: Raw data (CSV) from Amazon and Book-Crossing converted, via python scripts, in JSON for MongoDB and CSV files for populate Neo4j Graph. The scripts clean, filter and merge thousands of records to create profiles for books and users. What's more simulates a realistic social graph by deriving user connections and preferences directly from their reading history

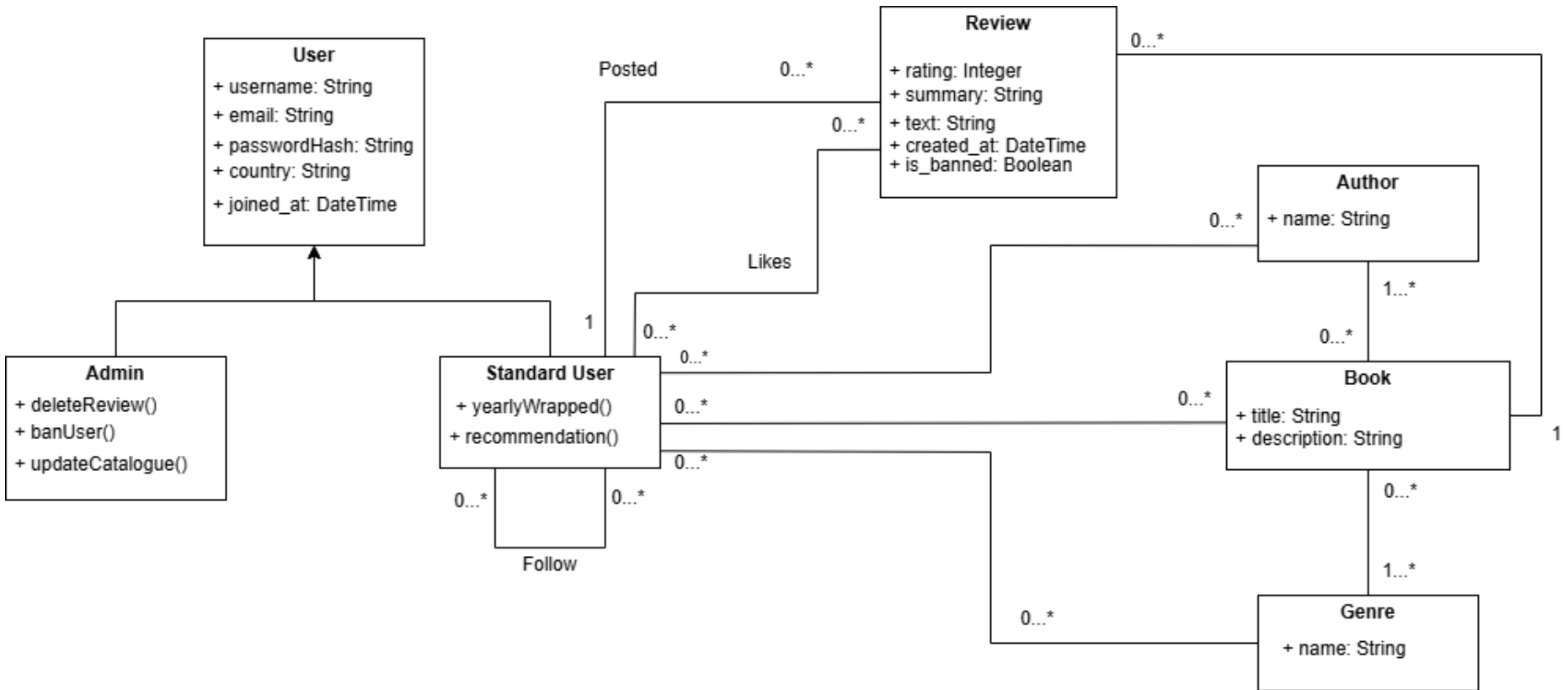
Dataset Description(2)

- **Volume**

MongoDB (81 MB)	Neo4j CSV File (29 MB)
• 27.7MB for Books • 3.5MB for Authors • 37.1MB for Reviews • 12.4MB for Users	• 4.32MB for nodes • 24.9MB for relations

- **Variety:** High data variety given a lot of different reviews coming from different sources
- **Velocity:** Old reviews are excluded from trending and monthly analytics, satisfying the temporal relevance requirement.

UML Design Class Diagram



Application non-functional requirements

Performance

- The system must efficiently process read-heavy workloads (*approx. 80% reads*).
- The system must minimize latency by leveraging Eventual Consistency between databases.

Reliability

- The system must implement redundancy strategies to prevent permanent data loss.

Usability

- The System must be highly available and fault-tolerant to ensure continuous service.
- The System must follow RESTful design principles for API communication

Security

- The system must enforce stateless authentication using JSON Web Tokens (JWT).
- The system must encrypt the user password.

Extensibility

- The system must support possible horizontal scaling via sharding strategies.

Error Handling

- The system must capture errors globally to return standardized responses (RFC 7807).
- The system must implement compensating transactions to handle synchronization failures.

Document DB Design (1)

Book Collection

```
_id: ObjectId('698db76b4a1f5c8fa49337a2')
title: "Pendragon (The Guide to the Territories of Halla )"
publication_year: 2000
description: "Conveys the sights of Halla from Bobby Pendragon's perspective, descri-"
  author: Object
    id: ObjectId('698db76b4a1f5c8fa49337a3')
    name: "D.J. Machale"
  genres: Array (1)
    0: "Juvenile Fiction"
  availability: "ACTIVE"
  external_ids: Object
    isbns: Array (3)
      0: "051513225X"
      1: "0739423738"
      2: "0380717573"
  recent_reviews_snapshot: Array (1)
    0: Object
      _id: ObjectId('698db7bf4a1f5c8fa49448c4')
      user_id: ObjectId('698db7854a1f5c8fa493b4a0')
      username: "User_A3GL86U3XW71EQ"
      rating: 32
      summary: "Not up to the Same Standard as the Series"
      date: 2017-05-07T02:00:00.000+00:00
  popular_reviews_snapshot: Array (1)
    0: Object
      _id: ObjectId('698db7bf4a1f5c8fa49448c4')
      user_id: ObjectId('698db7854a1f5c8fa493b4a0')
      username: "User_A3GL86U3XW71EQ"
      rating: 32
      num_of_like: 3
      summary: "Not up to the Same Standard as the Series"
      date: 2017-05-07T02:00:00.000+00:00
  stats_per_year: Array (1)
    0: Object
      year: 2017
      ratings_count: 1
      sum_rating: 32
  review_ids: Array (1)
    0: ObjectId('698db7bf4a1f5c8fa49448c4')
  month_score: Object
    rating_count: 1
    sum_rating: 32
    Current_Month: "2017-05"
  source: "amazon_master"
```

Document DB Design (2)

Author Collection

```
_id: ObjectId('698db76b4a1f5c8fa4934592')
name: "Edgar Allan Poe"
published_books: Array (9)
  0: Object
    _id: ObjectId('698db76b4a1f5c8fa4934591')
    title: "Eight Tales of Terror"
  1: Object
    _id: ObjectId('698db76b4a1f5c8fa493488f')
    title: "The Tell-Tale Heart"
  2: Object
    _id: ObjectId('698db76c4a1f5c8fa4935dae')
    title: "Complete Stories and Poems of Edgar Allan Poe"
  3: Object
    _id: ObjectId('698db76c4a1f5c8fa4936c8e')
    title: "Tales of Mystery and Imagination"
  4: Object
    _id: ObjectId('698db76c4a1f5c8fa49373bf')
    title: "Fall of the House of Usher and Other Stories (Signet Classics (Turtleb..."
  5: Object
    _id: ObjectId('698db76d4a1f5c8fa4937aef')
    title: "The Raven And Other Poems"
  6: Object
    _id: ObjectId('698db76e4a1f5c8fa49395df')
    title: "The Pit and the Pendulum"
  7: Object
    _id: ObjectId('698db76e4a1f5c8fa4939a9c')
    title: "The Narrative of Arthur Gordon Pym of Nantucket (Modern Library Classi..."
  8: Object
    _id: ObjectId('698db76e4a1f5c8fa493a884')
    title: "Edgar Allan Poe Reader (Courage Literary Classics)"
ratings_count: 14
sum_ratings: 1234
status: "ACTIVE"
```

Document DB Design (3)

Users Collection

```
_id: ObjectId('698db7854a1f5c8fa493aee8')
username : "nicktam36/"
password_hashed : "4e8d7994024d6d1aea57dacad08c986989b4e16aff09bdcfc8e52f90eda2c5bb/"
email : "garibals234@gmail.com/"
country : "US /"
joined_at : 2023-01-27T00:00:00.000+00:00
status : "active/"
bookshelf : Array (1)
  ▾ 0: Object
    book_id : 698db76b4a1f5c8fa49332d1
    title : "OF MICE AND MEN/"
    status : "read/"
    added_at : 2023-03-13T00:00:00.000+00:00
    ▾ author : Object
      id : 698db76b4a1f5c8fa49332bb
      name : "John Steinbeck/"
    ▾ genres : Array (1)
      0: "Fiction/"
    reviews_year : Array (1)
      ▾ 0: Object
        id : 698db7b84a1f5c8fa493eab0
        rating : 96
        book : "OF MICE AND MEN/"
    review_ids : Array (1)
      0: 698db7b84a1f5c8fa493eab0
```

Document DB Design (4)

Reviews Collection

```
_id: ObjectId('698db7ba4a1f5c8fa4940732')
user_id: ObjectId('698db7854a1f5c8fa493b3bc')
username: "User_A3M77GPST0EICR"
rating: 84
source: "amazon"
text: "There are some good and bad things to say about this book. The first o..."
summary: "A good book, but not sensational"
created_at: 2013-12-21T01:00:00.000+00:00
likes_count: 0
▼ book_snapshot: Object
  title: "The Fellowship of the Ring"
  book_id: ObjectId('698db76b4a1f5c8fa49331a8')
▼ author_snapshot: Object
  id: ObjectId('698db76b4a1f5c8fa4933176')
  name: "J. R. R. Tolkien"
```

Document DB Queries (1)

Books Ranking

It filters the statistics by a specific year, aggregates the total ratings, and enforces a minimum threshold to exclude statistical outliers. Finally, it calculates the exact average and sorts the results.

```
db.books.aggregate([
  // 1. Initial Match
  {
    "$match": {
      "availability": "ACTIVE"
    }
  },
  // 2. Filter Array (Year Logic)
  {
    "$project": {
      "title": 1,
      "author": 1,
      "targetStat": {
        "$filter": {
          "input": "$stats_per_year",
          "as": "stat",
          "cond": { "$eq": ["$stat.year", 2023] }
        }
      }
    }
  },
  // 3. Extract Totals (Flattening)
  {
    "$project": {
      "title": 1,
      "author": 1,
      "totalRatings": { "$sum": "$targetStat.ratings_count" },
      "sumRating": { "$sum": "$targetStat.sum_rating" }
    }
  },
  // 4. Threshold Filter
  {
    "$match": { "totalRatings": { "$gt": 5 } }
  },
  // 5. Calculate Average & Rename fields for DTO
  {
    "$project": {
      "name": "$title",
      "additionalInfo": "$author.name",
      "totalRatings": 1,
      "averageRating": { "$divide": ["$sumRating", "$totalRatings"] }
    }
  },
  // 6. Final Sort & Limit
  { "$sort": { "averageRating": -1 } },
  { "$limit": 25 }
])
```

```
// 3. Extract Totals (Flattening)
{
  "$project": {
    "title": 1,
    "author": 1,
    "totalRatings": { "$sum": "$targetStat.ratings_count" },
    "sumRating": { "$sum": "$targetStat.sum_rating" }
  }
},
// 4. Threshold Filter
{
  "$match": { "totalRatings": { "$gt": 5 } }
},
// 5. Calculate Average & Rename fields for DTO
{
  "$project": {
    "name": "$title",
    "additionalInfo": "$author.name",
    "totalRatings": 1,
    "averageRating": { "$divide": ["$sumRating", "$totalRatings"] }
  }
},
// 6. Final Sort & Limit
{ "$sort": { "averageRating": -1 } },
{ "$limit": 25 }
])
```


Document DB Queries (2)

Yearly Wrapped

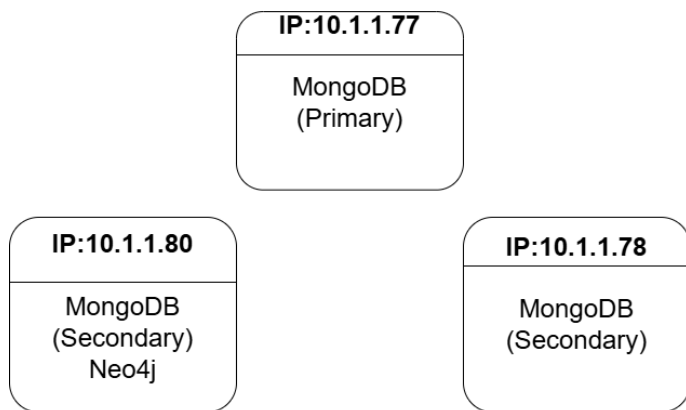
As you interact and rate, the platform aggregates your history to generate a personalized statistical summary at the end of the year—visualizing your top authors, favourite genres, and reading milestones.

```
db.users.aggregate([
  // 1. MATCH : Filter by specific user
  {
    "$match": { "_id": "CURRENT_USER_ID" }
  },
  // 2. ADDFIELDS : Pre-calculate data (filter and sort internal arrays)
  {
    "$addFields": {
      "best_book": {
        "$arrayElemAt": [
          {
            "$sortArray": {
              "input": { "$ifNull": ["$reviews_year", []] },
              "sortBy": { "rating": -1 } // DESC
            },
            0
          },
          "$yearly_books"
        ]
      },
      "worst_book": {
        "$arrayElemAt": [
          {
            "$sortArray": {
              "input": { "$ifNull": ["$reviews_year", []] },
              "sortBy": { "rating": 1 } // ASC
            },
            0
          },
          "$yearly_books"
        ]
      },
      "yearly_books": {
        "$filter": {
          "input": { "$ifNull": ["$bookshelf", []] },
          "as": "b",
          "cond": {
            "$and": [
              { "$eq": ["$b.status", "read"] },
              { "$gte": ["$b.added_at", ISODate("2026-01-01T00:00:00Z")] },
              { "$lte": ["$b.added_at", ISODate("2026-12-31T23:59:59Z")] }
            ]
          }
        }
      }
    }
  }
])
```

```
// 3. FACET : Execute 3 parallel sub-pipelines
{
  "$facet": {
    // Analysis 1: Top Authors
    "top_authors": [
      { "$project": { "yearly_books": 1 } },
      { "$unwind": "$yearly_books" },
      {
        "$group": {
          "_id": "$yearly_books.author.name",
          "count": { "$sum": 1 }
        }
      },
      { "$sort": { "count": -1 } },
      { "$limit": 3 }
    ],
    // Analysis 2: Top Genres
    "top_genres": [
      { "$project": { "yearly_books": 1 } },
      { "$unwind": "$yearly_books" },
      { "$unwind": "$yearly_books.genres" },
      {
        "$group": {
          "_id": "$yearly_books.genres",
          "count": { "$sum": 1 }
        }
      },
      { "$sort": { "count": -1 } },
      { "$limit": 3 }
    ],
    // Analysis 3: Metadata (Best / Worst & Totals)
    "meta": [
      {
        "$project": {
          "best_book": 1,
          "worst_book": 1,
          "total_books_read": { "$size": "$yearly_books" }
        }
      }
    ]
  }
})
```

MongoDB Replica Set Configuration

- **w=majority**: writes are acknowledged only after propagating to most nodes. This penalizes latency but favors data safety.
- **readPreference=secondaryPreferred**: the reads operation at first are performed on the Secondaries, if both are not available the read will fall back to the *Primary* node.
- **retryWrites = true**: If a write operation fails, the MongoDB driver will automatically attempt the write again *once*.



```
rsconf = {
  _id: "lsmdb",
  members: [
    { _id: 0, host: "10.1.1.77:27018", priority: 1 },
    { _id: 1, host: "10.1.1.78:27019", priority: 2 },
    { _id: 2, host: "10.1.1.80:27020", priority: 5 }
  ]
};
rs.initiate(rsconf);
```

MongoDB Indexes (1)

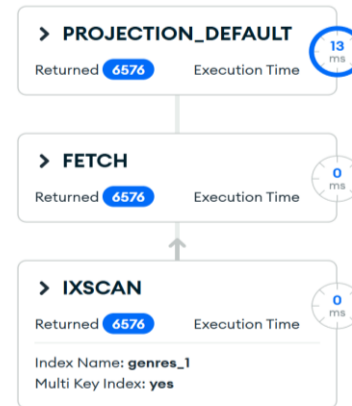
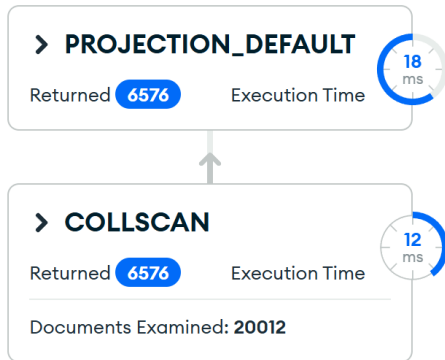
Collection	Field	Type
Books	genres	Regular(Asc)
Books	title	Text
Authors	name	Text

This indexes are designed to optimize some frequent operation like:

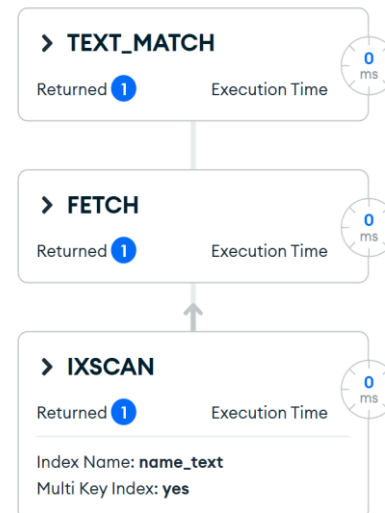
- **Ranking by genre**
- **Search by name/title**

MongoDB Indexes (2)

Index on book: Ranking by genre



Index on author.name: Find author by name



Discussion on MongoDB Data Sharding

How could sharding have been theoretically implemented?

To implement this, a shard key must be selected. The shard keys that could be chosen for this system are:

Reviews:id , Users:username.

- No cross-shards API for this keys
- Reviews and users are the most growing collections
- Even load balancing

Concerning the partitioning algorithm, the **consistent hashing algorithm** could be selected to ensure even distribution (we use alphanumeric id and unique username) and support nodes addition and removal.

Graph DB Design

Entities:

- User(mongold, username, country)
- Book(mongold, title, year).
- Review(mongold, rating, createdAt).
- Author
- Genre

Relationships:

- **FOLLOWS**: (User, User)
- **LIKES**: (User, Book)
- **POSTED**: (User, Review)
- **LIKES**: (User, Review)
- **LIKES**: (User, Author)
- **LIKES**: (User, Genre)
- **REFER_TO**: (Review, Book)
- **BELONGS_TO**: (Book, Genre)
- **WROTE**: (Author, Book)



Graph-based queries (1)

User Recommendations

```
@Query("""
MATCH (u:User {mongoId: $userId})
OPTIONAL MATCH (u)-[:FOLLOWS]→(:User)-[:LIKES]→(b1:Book)
OPTIONAL MATCH (u)-[:LIKES]→(:Author)-[:WROTE]→(b2:Book)
OPTIONAL MATCH (u)-[:LIKES]→(:Genre)←[:BELONGS_TO]-(b3:Book)
WITH collect(b1) + collect(b2) + collect(b3) AS recommendations, u
UNWIND recommendations AS book
WITH u, book
WHERE NOT EXISTS((u)-[:POSTED]→(:Review)-[:REFER_TO]→(book)) AND book IS NOT NULL
WITH book, count(*) AS score
ORDER BY score DESC
LIMIT $limit
MATCH (a:Author)-[:WROTE]→(book)
RETURN book.mongoId AS bookId,
       book.title AS title,
       book.year AS publicationYear,
       score,
       collect(a.name) AS authors // Uso collect per evitare righe duplicate se ci sono più autori
""")
List<RecommendationProjection> getUserRecommendations(@Param("userId") String userId, @Param("limit") int
limit);
```

The recommendation of books using liked by the users, current follows, authors the user likes, and genres the user likes. To optimize the recommendation engine and prevent suggesting books the user has already read, a pre-query is executed on the MongoDB backend to extract the read books from the user's bookshelf.

Graph-based queries (2)

Internationality Index

```
@Query("""
MATCH (b:Book {mongoId: $bookId})
OPTIONAL MATCH (b)←[:REFER_TO]-(r:Review)←[:POSTED]-(reviewer:User)
OPTIONAL MATCH (b)←[:LIKES]-(liker:User)
WITH reviewer, liker
WITH collect(DISTINCT reviewer) + collect(DISTINCT liker) AS users
UNWIND users AS u
WITH u WHERE u IS NOT NULL AND u.country IS NOT NULL
RETURN u.country AS country, count(DISTINCT u.mongoId) AS uniqueUsers, count(*) AS totalInteractions
ORDER BY uniqueUsers DESC
""")
List<InternationalityProjection> calculateBookInternationality(@Param("bookId") String bookId);
```

This analytic computes the “global reach” (Internationality Index) of a Book or an Author by analyzing the geographic origin (country property) of users who interact with the entity. The traversal captures both users who posted a review and users who simply left a “Like”

Graph-based queries (3)

Genre Influencer

```
@Query("""
MATCH (g:Genre {name: $genreName})←[:BELONGS_TO]-(b:Book)←[:REFER_TO]-(r:Review)←[:POSTED]-(influencer:User)
MATCH (r)←[:LIKES]-(fan:User)
WITH influencer,
     count(DISTINCT r) AS numReviews,
     count(fan) AS totalLikes
WHERE numReviews > 3 AND influencer.username <> ""
RETURN influencer.username AS username,
       totalLikes AS totalEngagement,
       numReviews AS numReviews,
       toFloat(totalLikes) / numReviews AS avgLikesPerReview
ORDER BY avgLikesPerReview DESC
LIMIT $limit
""")
List<InfluencerProjection> findGenreInfluencers(@Param("genreName") String genreName, @Param("limit") int limit);
```

It searches for users who wrote reviews that received a high volume of "Likes" from other users, either globally or filtered by a specific genre. To prevent anomalies, the query enforces a minimum threshold of reviews and excludes soft-deleted or banned users (represented by an empty string username).

Neo4j Indexes

Example: Find user by ID

The images on the bottom illustrate what happens when the **Find User** is performed with or without using the index (same for the other nodes search)

Collection	Field
User	mongoid
Books	mongoid
Authors	mongoid
Review	mongoid
Genre	name

The indexes help to locate the starting Node of a query, so common sense is to make them on the ids of the nodes

NodeByLabelScan@booksphere
u
u:User
248 memory (bytes) 318 pagecache hits 0 pagecache misses 12.006 estimated rows 54.060 time (ms)
12.007 db hits

NodeUniqueIndexSeek@booksphere
u
UNIQUE u:User(mongoid) WHERE mongoid = \$autostring_0
248 memory (bytes) 23 pagecache hits 2 pagecache misses 1 estimated rows 2 db hits 1.553 time (ms)
2 db hits

Handling Intra-DB Consistency

In **Neo4J** the consistency is guaranteed by the atomicity of the CRUD operations inside the single database replica.

In MongoDB, consistency is guaranteed atomically at the single-document level. However, when **data denormalization** is employed, maintaining consistency across redundant data must be managed at the application level, either **Synchronously** (Strict Consistency) or **Asynchronously** (Eventual Consistency).

For instance, we leverage the **@Async** annotation to update pre-computed aggregations, such as the stats_per_year bucket, immediately after a new review is posted.

Handling Inter-DB Consistency

To ensure **Inter-Database Consistency** in a Polyglot Architecture, we implement either **Strict** or **Eventual Consistency** strategies depending on the criticality of the operation.

Strict Consistency

Adopted for **critical operations** where data integrity is paramount, such as **User Registration** or **Catalog Management** (e.g., adding a new Book).

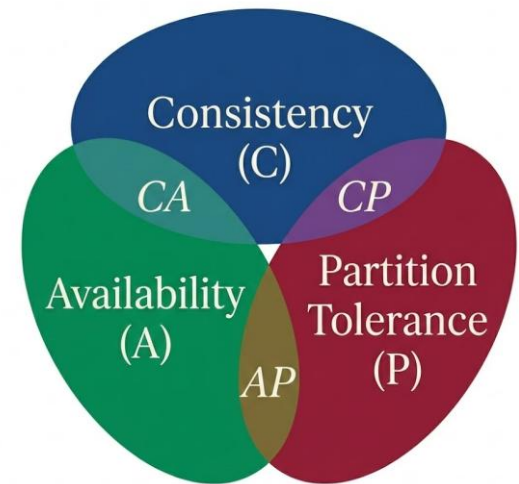
Eventual Consistency

We apply this strategy for frequent operations to ensure **low latency**. A key example is the eventual consistency update of the `popular_reviews_snapshot` in the Book document whenever a review receives a new **Like**.

CAP Theorem

By default, a MongoDB replica set strongly favors CP. However, to enhance Availability for read-heavy operations:

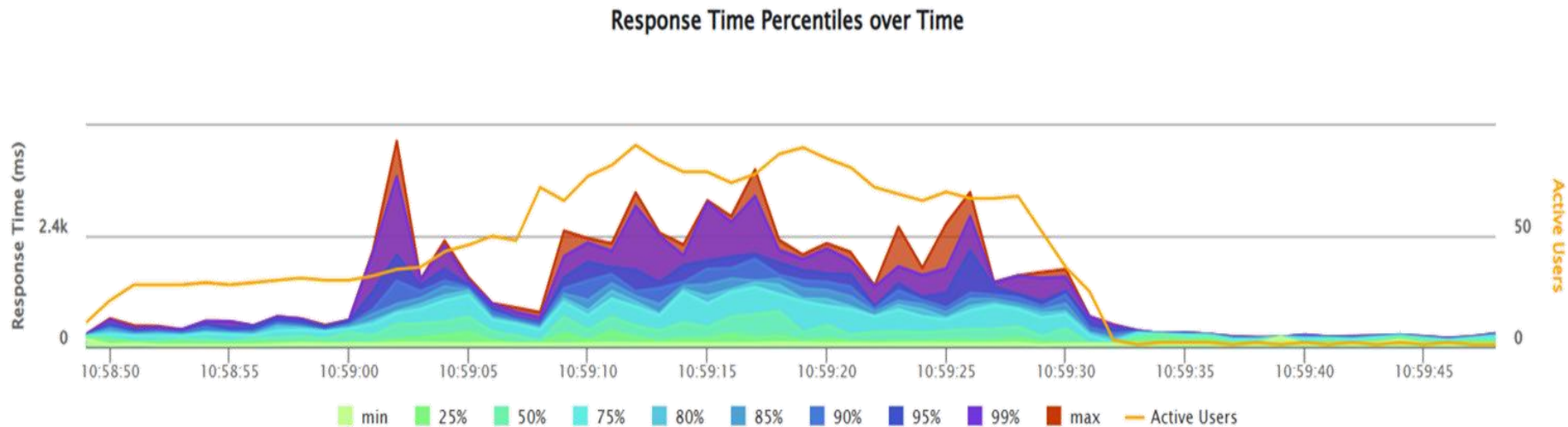
- Write Concern: penalizes latency but favors data safety (CP), with stale data (AP)
- Read Preference: AP model.



Test (gatling)

Spike Test

Response Time Impact During Traffic Surge and Recovery



Swagger UI REST APIs documentation

Follow (Registered) User follow/unfollow endpoints	
POST	/api/v1/me/follow Follow a user
GET	/api/v1/me/friends Get followed users
DELETE	/api/v1/me/unfollow/{userId} Unfollow a user
Analytics (Open) Public analytics and statistics endpoints	
GET	/api/v1/analytics/rankings/trendingbooks Get trending books
GET	/api/v1/analytics/rankings/bybookids Get book rankings by book IDs
GET	/api/v1/analytics/rankings/authors Get author rankings
GET	/api/v1/analytics/internationality/{entityId} Calculate Internationality Index
GET	/api/v1/analytics/influencers Identify genre influencers
GET	/api/v1/analytics/books Get book rankings
GET	/api/v1/analytics/books/revaluated Get book revaluation
Admin - Books Admin endpoints for book catalog management	
PUT	/api/v1/admin/books/{id} Update book
DELETE	/api/v1/admin/books/{id} Delete book
POST	/api/v1/admin/books Add new book
Reviews (Registered) Review management endpoints for registered users	
POST	/api/v1/me/reviews Create a review
DELETE	/api/v1/me/reviews/{reviewID} Delete a review
PATCH	/api/v1/me/reviews/{reviewID} Update a review
User Features (Registered) Personalized recommendations and yearly recap	
GET	/api/v1/me/wrapped Get yearly wrapped
GET	/api/v1/me/recommendations Get personalized recommendations
Books (Open) Public book endpoints accessible to all users	
GET	/api/v1/books Search book by title
GET	/api/v1/books/{id} Get book by ID
Users (Open) Public user profile endpoints	
GET	/api/v1/users/username/{username} Get user by username
GET	/api/v1/users/id/{id} Get user by ID
Authentication (Open) User authentication endpoints	
POST	/api/v1/auth/register Register new user
POST	/api/v1/auth/login User login

Admin - Catalog Admin endpoints for author and genre management	
PUT	/api/v1/admin/authors/{id} Update author
DELETE	/api/v1/admin/authors/{id} Delete author
POST	/api/v1/admin/genres Add new genre
POST	/api/v1/admin/authors Add new author
Reviews (Open) Public review endpoints accessible to all users	
GET	/api/v1/reviews Get reviews by IDs
Profile (Registered) User profile management endpoints	
PATCH	/api/v1/me/username Update username
DELETE	/api/v1/me/account Delete account
Bookshelf (Registered) User bookshelf management endpoints	
POST	/api/v1/me/bookshelf Add book to bookshelf
PATCH	/api/v1/me/bookshelf/{bookID} Update book status
DELETE	/api/v1/me/bookshelf/{bookID} Remove book from bookshelf
Authors (Open) Public author endpoints accessible to all users	
GET	/api/v1/authors Search author by name
GET	/api/v1/authors/{id} Get author by ID
Admin - Moderation Admin endpoints for content moderation and user management	
PATCH	/api/v1/admin/users/{id}/ban Ban user
GET	/api/v1/admin/users View all users
Likes (Registered) Like/unlike endpoints for books, reviews, authors, and genres	
POST	/api/v1/me/likes/genres Like a genre
POST	/api/v1/me/likes/authors Like an author
POST	/api/v1/me/like/review Like a review
POST	/api/v1/me/like/book Like a book
GET	/api/v1/me/liked/review Get liked reviews
GET	/api/v1/me/liked/genre Get liked genres
GET	/api/v1/me/liked/book Get liked books
GET	/api/v1/me/liked/author Get liked authors
DELETE	/api/v1/me/unlikes/review/{reviewID} Unlike a review
DELETE	/api/v1/me/unlike/genres/{name} Unlike a genre
DELETE	/api/v1/me/unlike/book/{bookID} Unlike a book
DELETE	/api/v1/me/unlike/authors/{authorID} Unlike an author

Swagger UI REST APIs documentation

Find book by title

GET /api/v1/books Search book by title

Search for books by title (supports partial matching). Returns paginated results.

Parameters

Name	Description
title	Book title to search
string (query)	The Fellowship
page	Page number (0-indexed)
integer(int32) (query)	0
size	Page size
integer(int32) (query)	20

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/v1/books?title=TheFellowship&page=0&size=20' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/v1/books?title=TheFellowship&page=0&size=20
```

Server response

Code Details

200

Response body

```
{
  "content": [
    {
      "id": "696db7db41f5c8fa9331a8",
      "title": "The Fellowship of the Ring",
      "publicationYear": 2000,
      "description": "Begin your Journey into Middle-earth... The inspiration for the upcoming original series on Prime Video, The Lord of the Rings: The Rings of Power, The Fellowship of the Ring is the first part of J.R.R. Tolkien's epic adventure The Lord of the Rings. One Ring to rule them all, One Ring to find them, One Ring to bring them all and in the darkness bind them. Sauron, the Dark Lord, has gathered to him all the Rings of Power—the means by which he intends to rule Middle-earth. All he lacks is his plan."
    }
  ],
  "pages": {
    "id": "696db7db41f5c8fa9331a8",
    "name": "The Fellowship of the Ring",
    "publishedBooks": null,
    "currentPage": null,
    "ratingCount": null,
    "numberOfPages": null
  },
  "pages": [
    "fiction"
  ],
  "books": [
    "696db7db41f5c8fa9331a8",
    "696db7db41f5c8fa9331a8",
    "696db7db41f5c8fa9331a8"
  ]
}
```

Code 200: book found

Add a book to the bookshelf

POST /api/v1/me/bookshelf Add book to bookshelf

Add a book to the user's bookshelf with a specific status (to_read, reading, read)

Parameters

No parameters

Request body *required*

application/json

```
{
  "bookId": "696db7db41f5c8fa9331a8",
  "status": "to_read"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/v1/me/bookshelf' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "bookId": "696db7db41f5c8fa9331a8",
    "status": "to_read"
  }'
```

Request URL

```
http://localhost:8080/api/v1/me/bookshelf
```

Server response

Code Details

401

Error: response status is 401

Response body

```
{
  "error": "Unauthorized",
  "message": "Full authentication is required to access this resource"
}
```

Code 401: user not authenticated

Live Demo with Postman

